

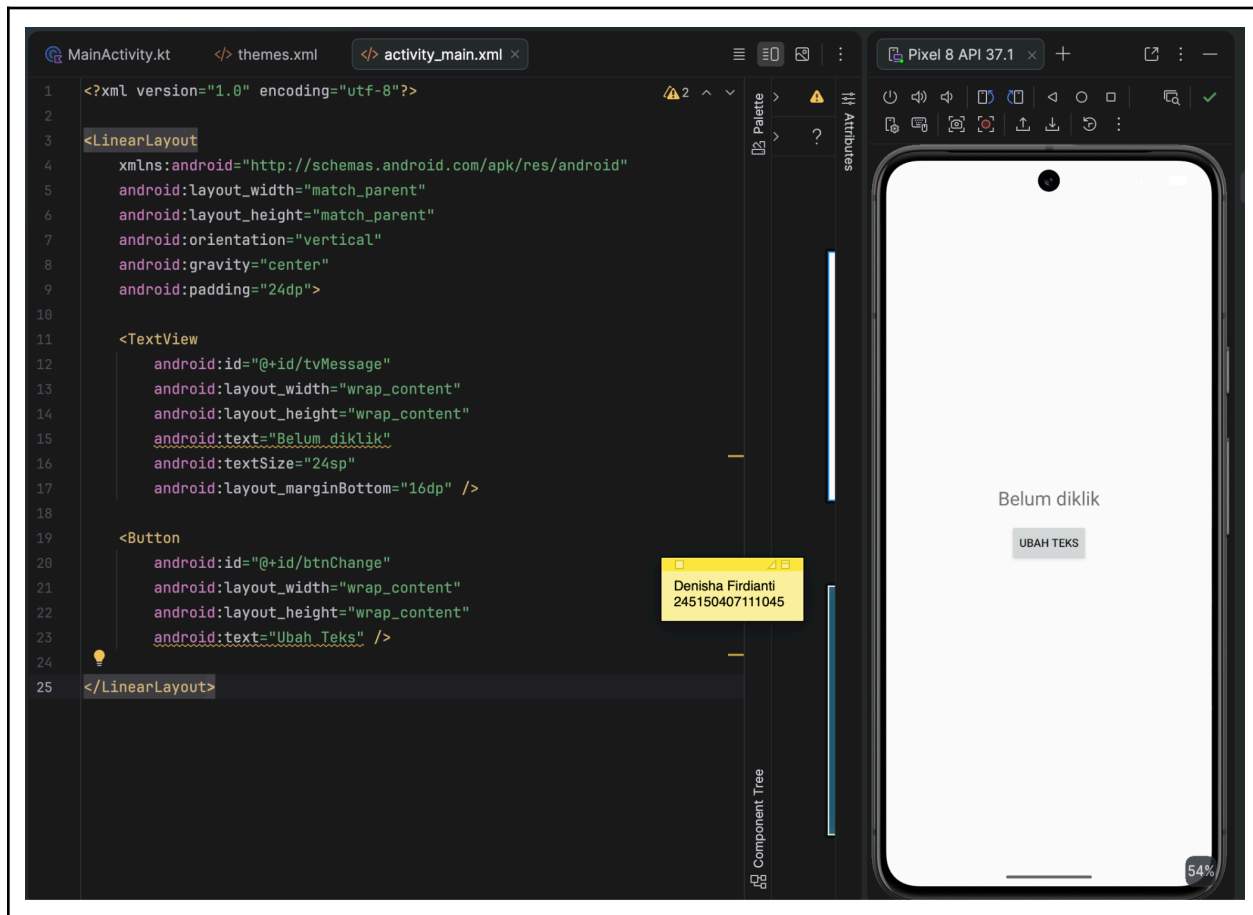
Nama : Denisha Firdianti

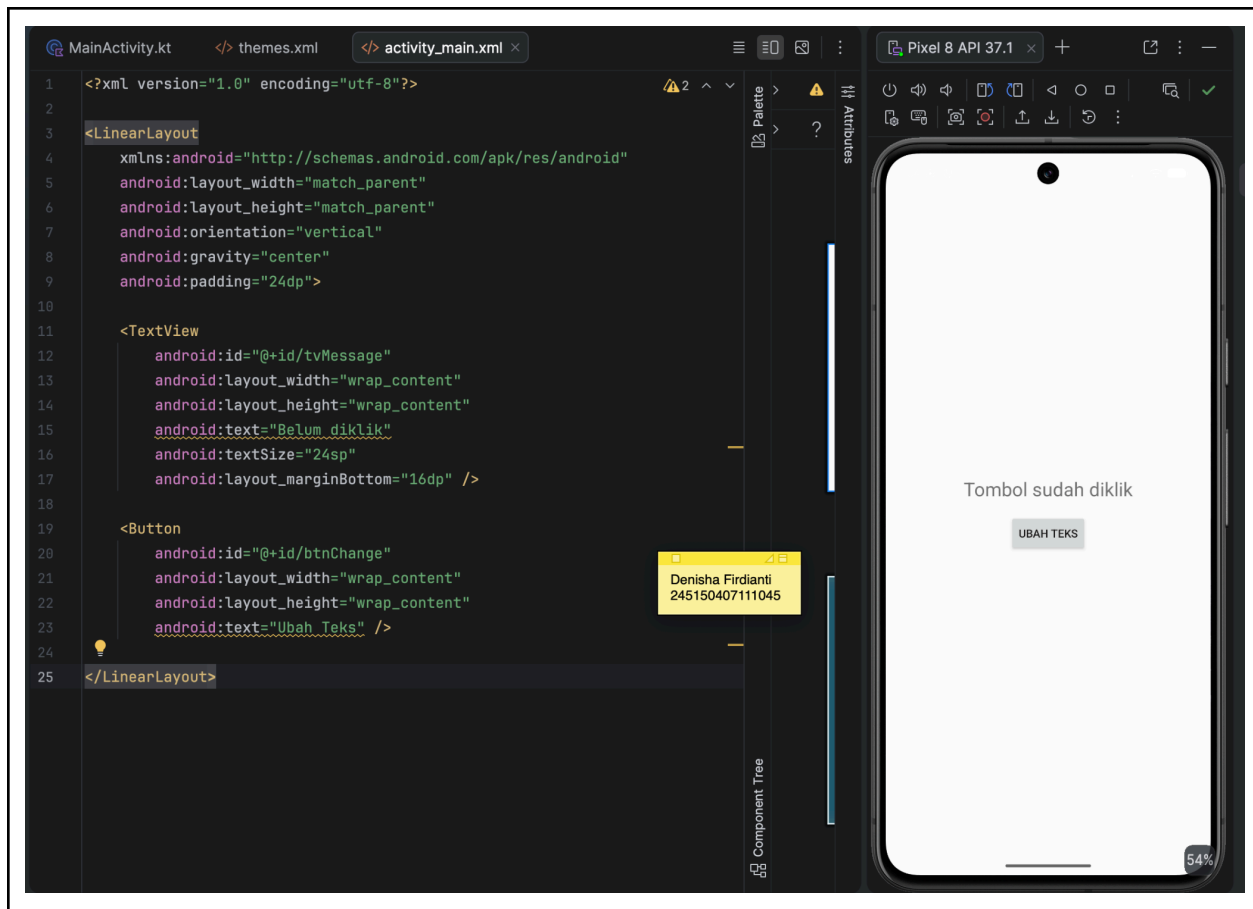
NIM : 245150407111045

MODUL 3- Event, State, Imperatif-Deklaratif, dan Implisit Intent

Aktivitas Percobaan 1

1. Jalankan kedua aplikasi dan ambil screenshot kondisi sebelum serta sesudah tombol diklik. Pada project XML/View, objek apa yang diubah secara langsung ketika tombol diklik? Tunjukkan baris kodenya.





- Pada project XML/View, objek yang diubah secara langsung adalah `TextView` dengan id `tvMessage`. Baris kodenya adalah `tvMessage.text = "Tombol sudah diklik"`.
3. Pada project Compose, nilai apa yang diubah ketika tombol diklik? Tunjukkan baris kodenya.
- Pada project Compose, nilai yang diubah ketika tombol diklik adalah state message, baris kodenya adalah `message = "Tombol sudah diklik"`.
4. Mengapa pada Compose tidak terdapat perintah seperti `tvMessage.text` atau `setText()`?
- Karena Compose menggunakan pendekatan deklaratif, jadi tidak mengubah View secara langsung, tetapi mengubah state, setelah state berubah, Compose memperbarui tampilan melalui recomposition.
5. Jelaskan alur perubahan pada XML/View dengan pola: event -> View -> perubahan tampilan.
- Pada XML/View, alurnya adalah event -> View -> perubahan tampilan. Ketika tombol diklik, event menjalankan kode yang mengubah `tvMessage`, lalu tampilan `TextView` berubah.
6. Jelaskan alur perubahan pada Compose dengan pola: event -> state -> recomposition -> perubahan tampilan.

- Pada Compose, alurnya adalah event -> state -> recomposition -> perubahan tampilan, lalu ketika tombol diklik, state message berubah, kemudian Compose melakukan recomposition dan menampilkan teks terbaru.

7. Apa yang terjadi jika message diganti menjadi variabel biasa: `var message = "Belum diklik"`? Jelaskan penyebabnya.

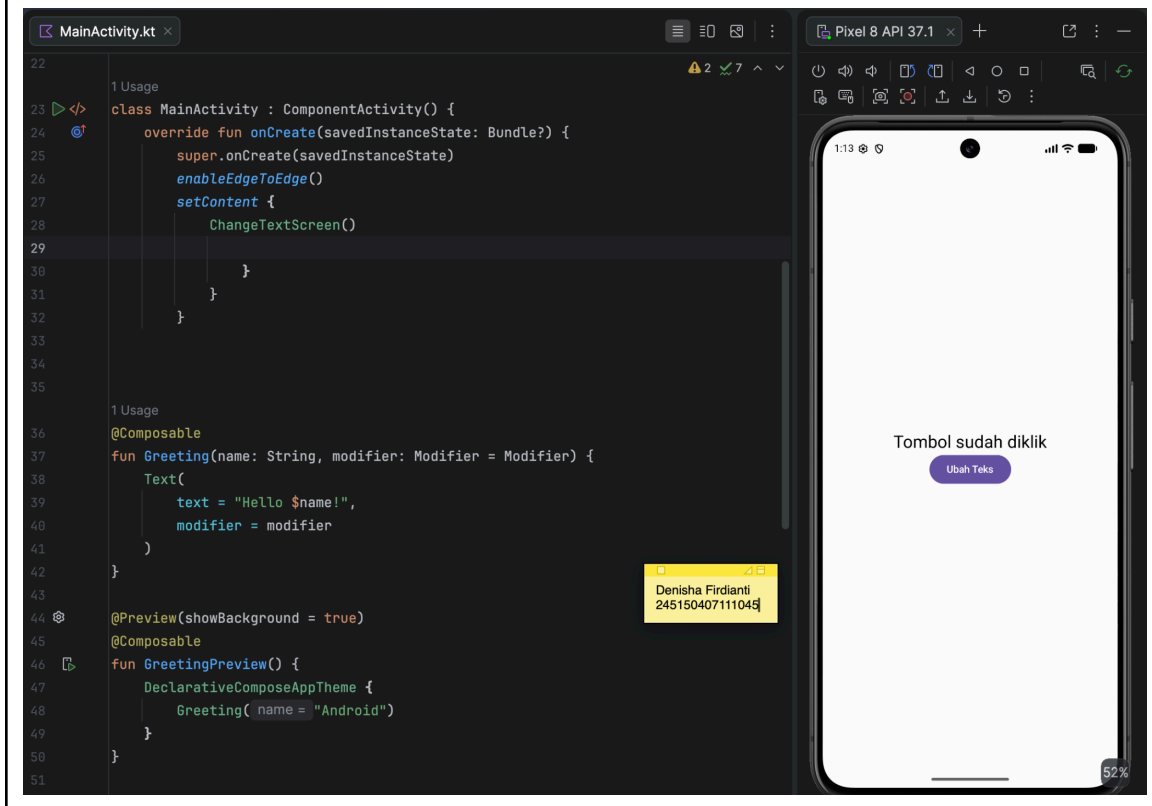
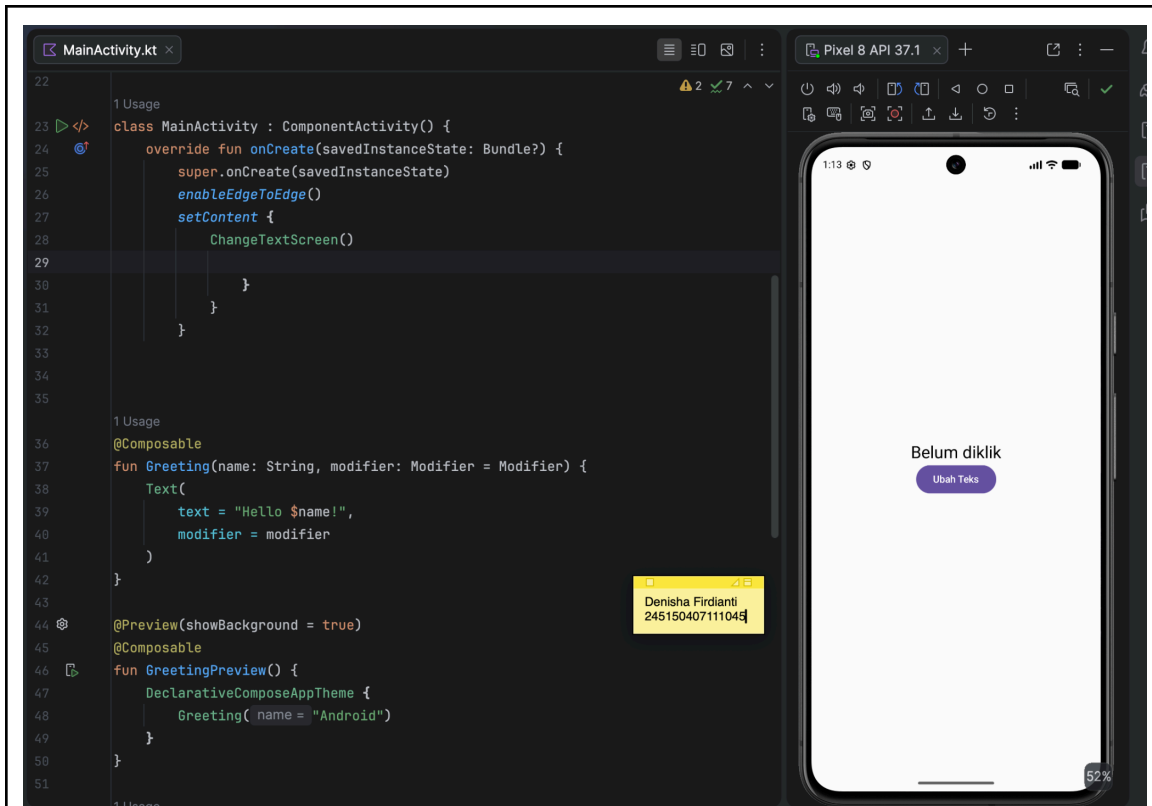
- Jika message hanya dibuat sebagai `var message = "Belum diklik"`, perubahan nilainya tidak akan membuat Compose melakukan recomposition, maka, teks pada UI tidak otomatis berubah karena message bukan state yang diamati Compose.

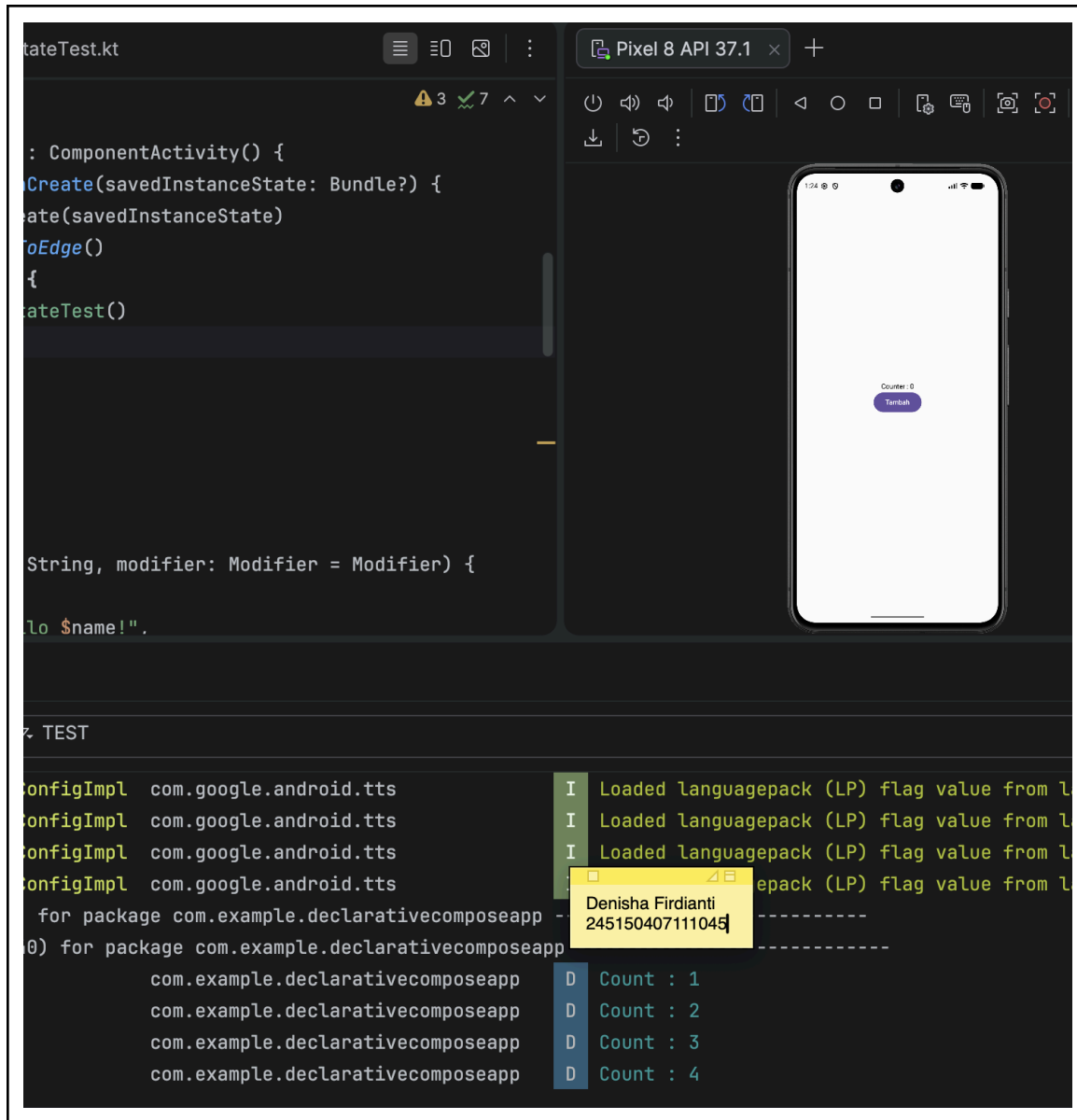
8. Tuliskan kesimpulan perbedaan cara berpikir imperatif dan deklaratif menggunakan kalimat Anda sendiri.

- Perbedaan cara berpikirnya adalah pendekatan imperatif berfokus pada bagaimana mengubah tampilan secara langsung, sedangkan pendekatan deklaratif berfokus pada menentukan tampilan berdasarkan state, jadi, pada Compose kita mengubah state dan membiarkan Compose memperbarui tampilan.

Aktivitas Percobaan 2

1. Jalankan program, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian buka LogCat, filter LogCat dengan tulisan TEST. Kemudian klik tombol "Tambah" beberapa kali pada aplikasi, ambil satu screenshot tampilan aplikasi setelah tombol "Tambah" diklik beberapa kali.





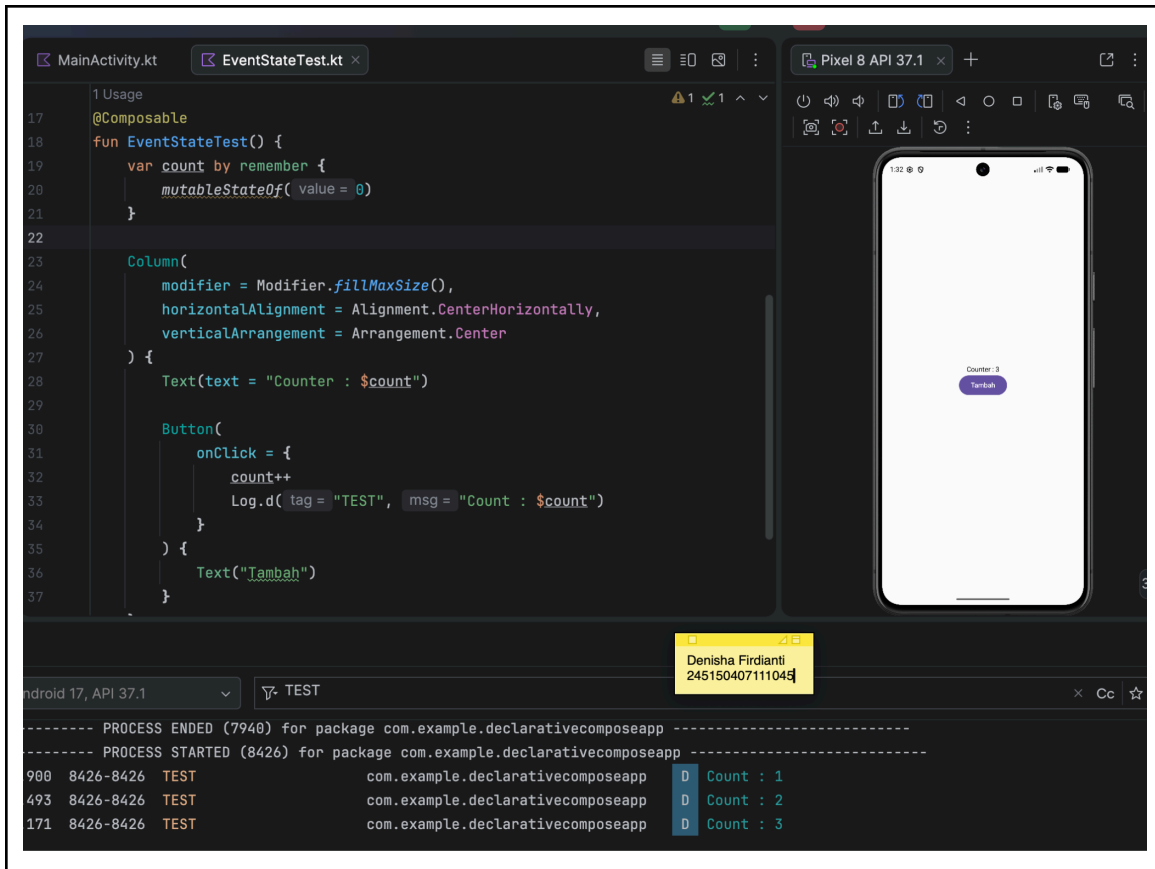
2. Apa yang muncul di LogCat setiap kali tombol "Tambah" diklik?
 - LogCat menampilkan pesan Count : 1, Count : 2, Count : 3, dan seterusnya sesuai jumlah klik pada tombol.
3. Apa yang terjadi pada tampilan aplikasi setiap kali tombol "Tambah" diklik?
 - Tampilan Counter tetap menunjukkan Counter : 0, meskipun tombol sudah diklik beberapa kali.
4. Apa yang menyebabkan hal tersebut terjadi?
 - Karena count masih berupa variabel biasa (`var count = 0`), bukan state Compose. Perubahannya tidak membuat Compose melakukan recomposition.
5. Langkah selanjutnya adalah mengubah kode sebelumnya untuk menggunakan state yaitu pada Composable EventStateTest. Ubah kode program pada bagian fungsi Composable

EventStateTest dari yang sebelumnya: `var count = 0`, menjadi: `var count by remember { mutableStateOf(0) }`. Berikut kode lengkap EventStateTest:

```
@Composable
fun EventStateTest() {
    var count by remember { mutableStateOf(0) }
    Column(
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally,
        modifier = Modifier.fillMaxSize()

    ) {
        Text("Counter : $count")
        Button(onClick = {
            count++
            Log.d("TEST", "Counter berubah : $count")
        }) {
            Text("Tambah")
        }
    }
}
```

Jalankan program kembali, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian buka LogCat, filter LogCat dengan tulisan TEST. Kemudian klik tombol "Tambah" beberapa kali pada aplikasi, ambil satu screenshot tampilan aplikasi setelah tombol "Tambah" diklik beberapa kali.



6. Apa yang muncul di LogCat setiap kali tombol "Tambah" diklik?
 - LogCat menampilkan Counter berubah : 1, Counter berubah : 2, Counter berubah : 3, dan seterusnya sesuai jumlah klik.
7. Apa yang terjadi pada tampilan aplikasi setiap kali tombol "Tambah" diklik?
 - Nilai Counter pada tampilan ikut bertambah, misalnya Counter : 1, kemudian Counter : 2, Counter : 3, dan seterusnya.
8. Apa yang menyebabkan hal tersebut terjadi dan berbeda dengan percobaan sebelumnya?
 - Karena count sekarang dibuat sebagai state menggunakan `remember { mutableStateOf(0) }`, jadi ketika nilainya berubah, Compose mengetahui perubahan tersebut dan memperbarui tampilan melalui recomposition.
9. Apa yang dimaksud recomposition pada Compose?
 - Proses Compose menjalankan kembali bagian UI yang bergantung pada state yang berubah sehingga tampilan diperbarui sesuai nilai state terbaru.
10. Langkah selanjutnya adalah menambah kode sebelumnya dengan menambah state baru yaitu text bertipe string pada Composable EventStateTest. Kemudian di dalam Column panggil composable OutlinedTextField di bawah Button Tambah. Penambahan kode program membuat kode pada fungsi Composable EventStateTest menjadi:

```
fun EventStateTest() {
  var count by remember { mutableStateOf(0) }
```

```

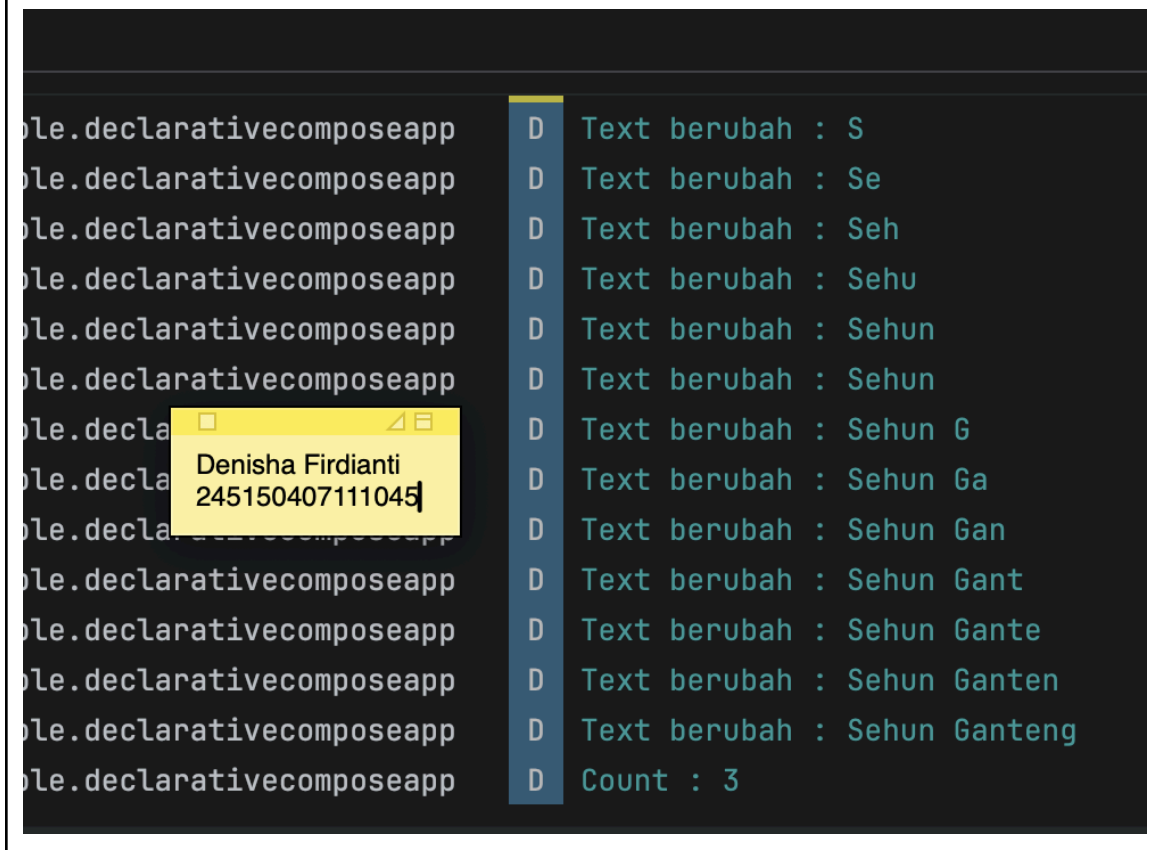
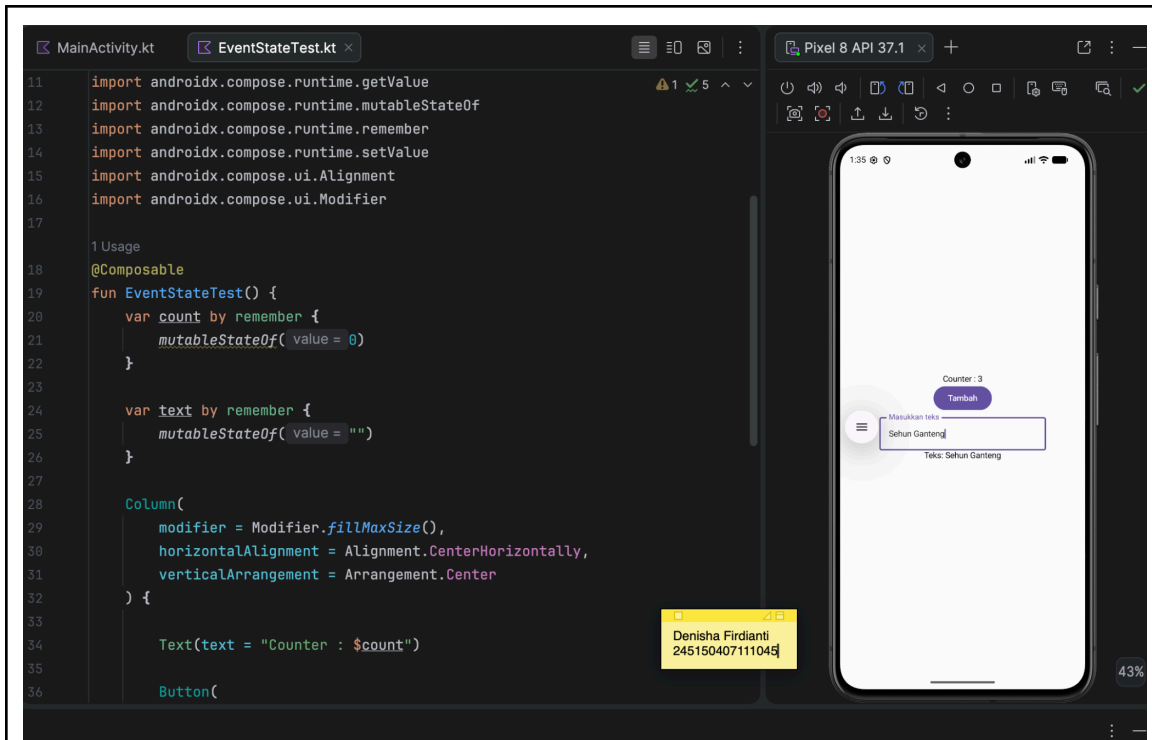
var text by remember { mutableStateOf("") }

Column(
    verticalArrangement = Arrangement.Center,
    horizontalAlignment = Alignment.CenterHorizontally,
    modifier = Modifier.fillMaxSize()

) {
    Text("Counter : $count")
    Button(onClick = {
        count++
        Log.d("TEST", "Counter berubah : $count")
    }) {
        Text("Tambah")
    }
    OutlinedTextField(
        value = text,
        onValueChange = {
            text = it
            Log.d("TEST", "Text berubah : $it")
        },
        label = { Text("Text Field") }
    )
}
}

```

Jalankan program kembali, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian buka LogCat, filter LogCat dengan tulisan TEST. Kemudian klik tombol "Tambah" beberapa kali pada aplikasi, ambil satu screenshot tampilan aplikasi setelah tombol "Tambah" diklik beberapa kali.



11. Apa yang muncul di LogCat setiap kali menginputkan huruf di Text Field?

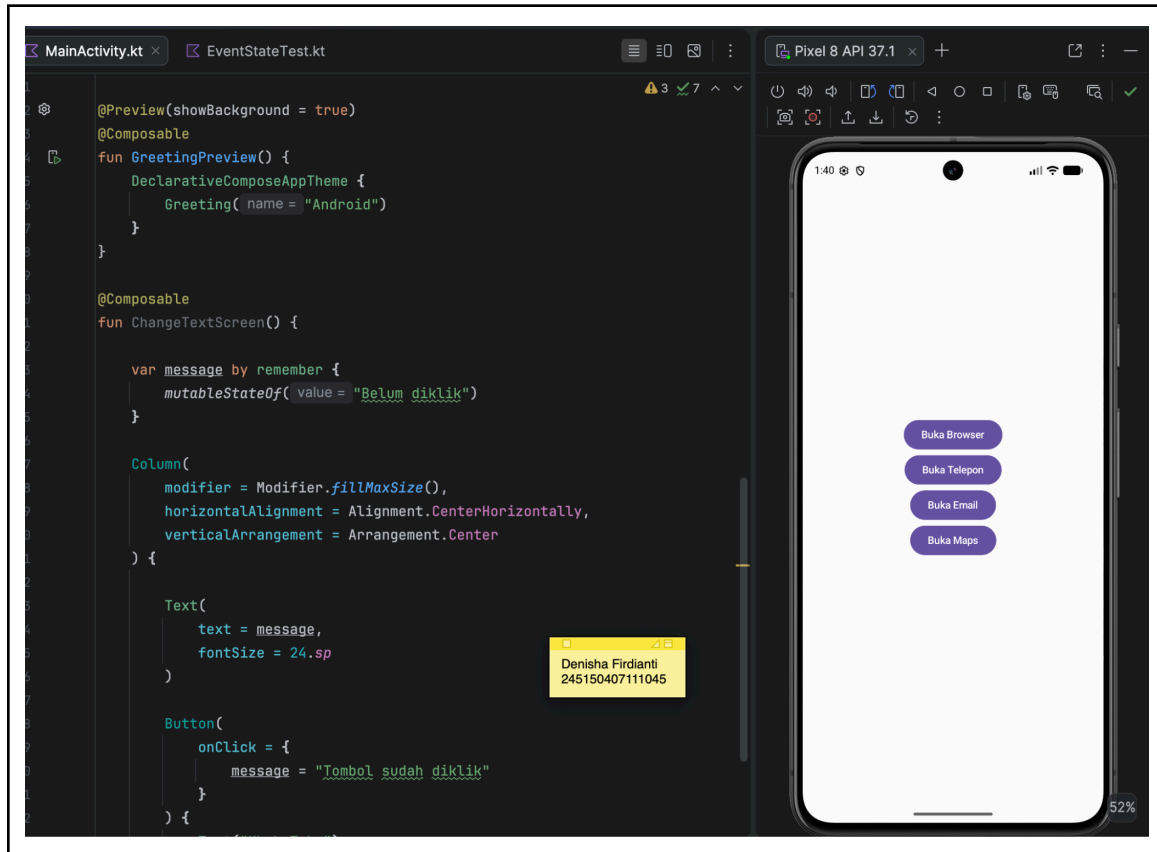
- LogCat menampilkan pesan Text berubah : diikuti teks yang sedang diketik dan pesan tersebut muncul setiap kali nilai Text Field berubah.
12. Apa yang terjadi pada tampilan aplikasi setiap kali menginputkan huruf di Text Field?
- Teks yang dimasukkan akan langsung tersimpan pada state text dan tampilan Text Field diperbarui mengikuti teks yang diketik.
13. Apa yang menyebabkan hal tersebut terjadi?
- Karena text dibuat sebagai state dengan remember { mutableStateOf("") } dan diperbarui melalui onValueChange. Perubahan state menyebabkan Compose melakukan recomposition.
14. Sebutkan state apa saja yang ada pada program yang terakhir dibuat?
- State yang digunakan adalah count bertipe angka untuk menyimpan jumlah klik dan text bertipe String untuk menyimpan teks yang dimasukkan pada Text Field.
15. Sebutkan event apa saja yang ada pada program yang terakhir dibuat?
- Event yang ada adalah event onClick pada tombol Tambah dan event onValueChange pada OutlinedTextField.
16. Kapan event - event tersebut dijalankan?
- Event onClick dijalankan ketika tombol Tambah ditekan, sedangkan onValueChange dijalankan ketika pengguna memasukkan atau mengubah teks pada Text Field.

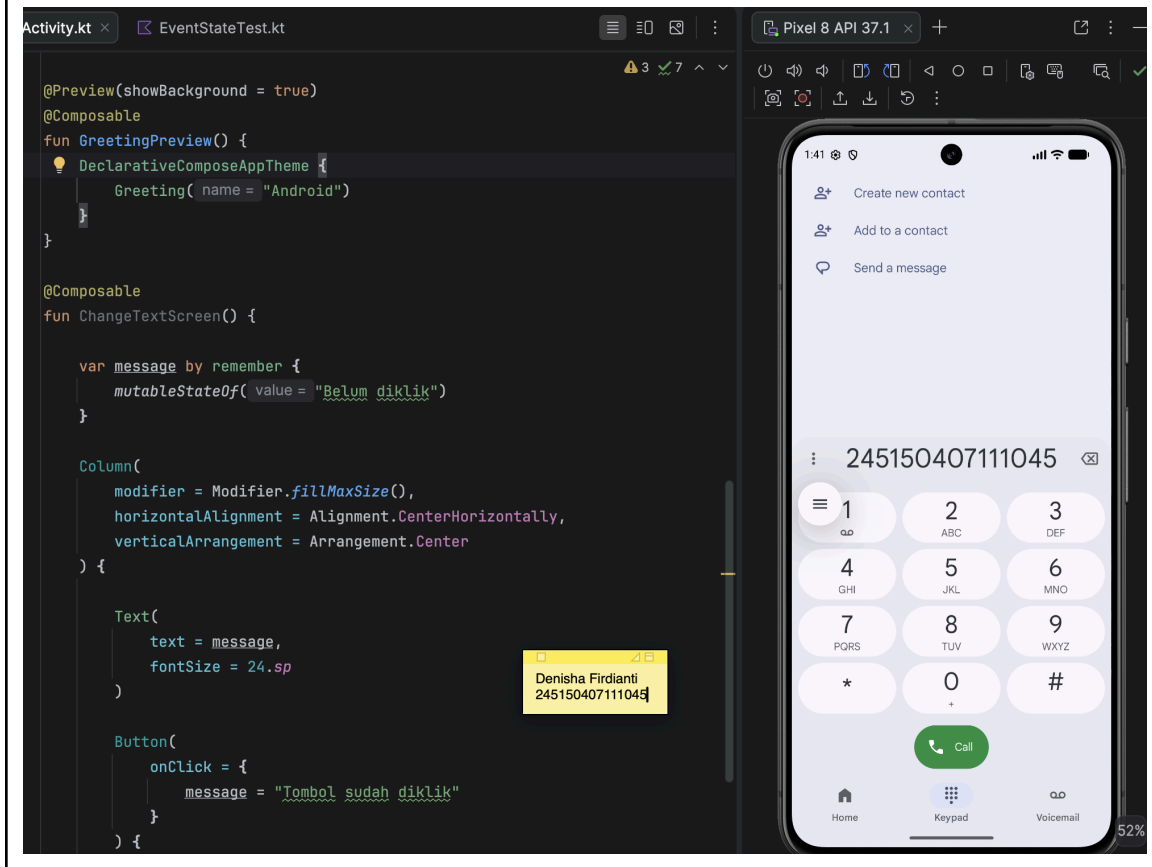
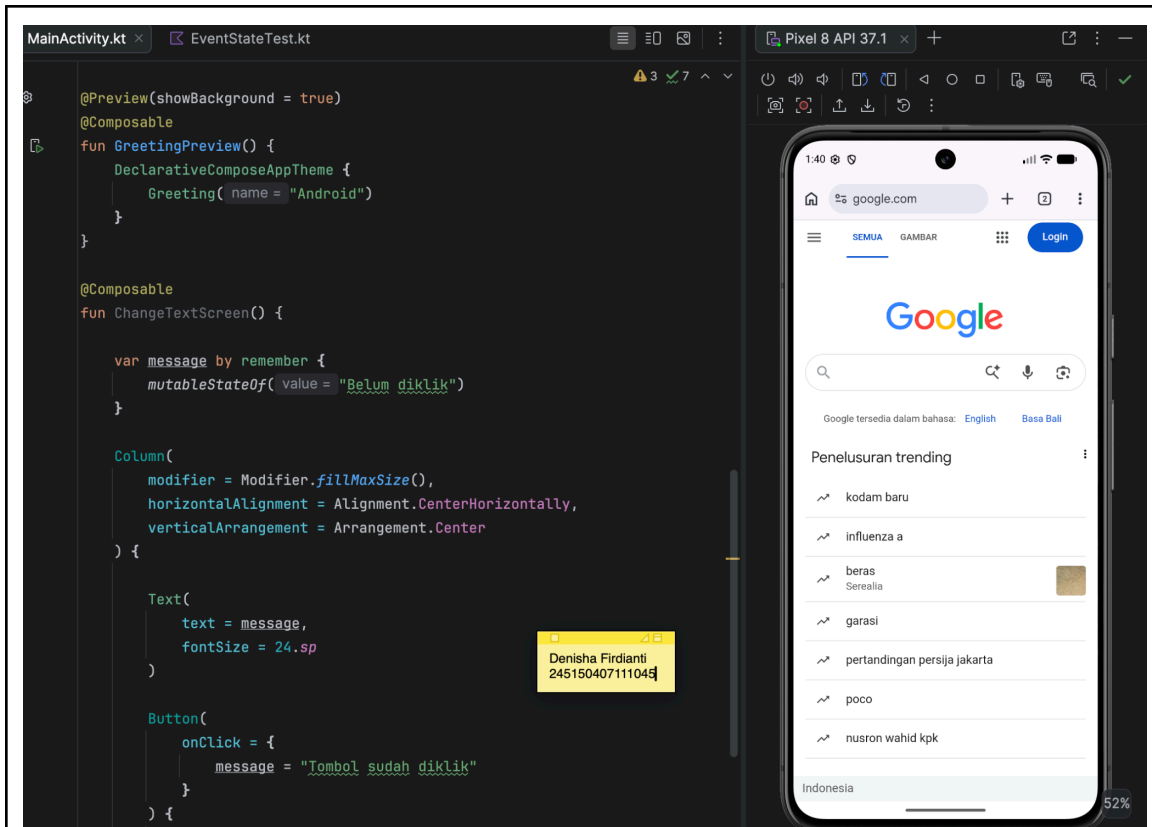
Aktivitas Percobaan 3

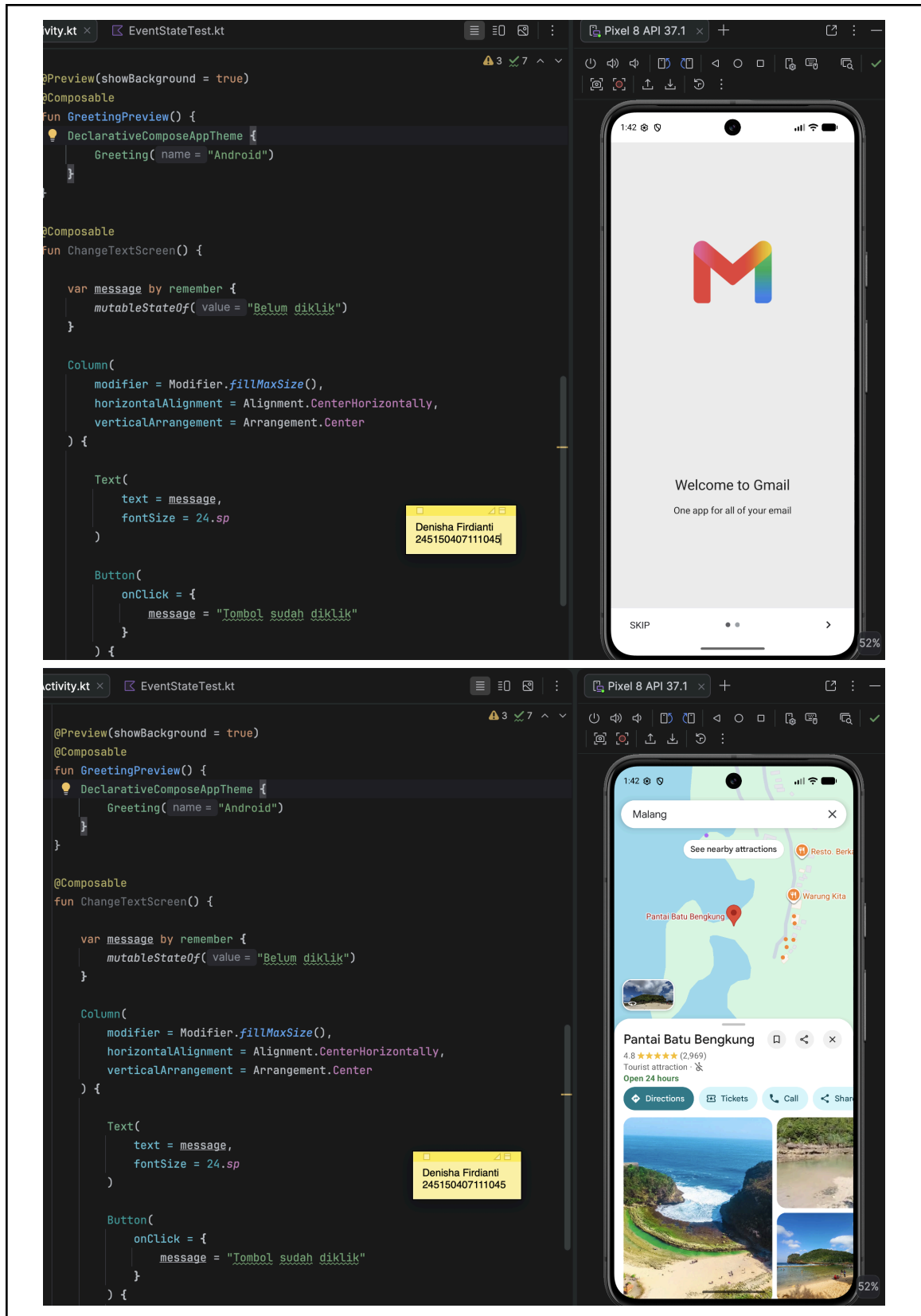
1. Jalankan program, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Lakukan pengujian pada setiap tombol:

- Klik Buka Browser
- Klik Buka Telepon
- Klik Kirim Email
- Klik Buka Maps

Amati apa yang terjadi pada setiap aksi, kemudian ambil screenshot hasilnya







2. Pada kode yang digunakan, terdapat beberapa action seperti:

`Intent.ACTION_VIEW`

`Intent.ACTION_DIAL`

`Intent.ACTION_SENDTO`

Apa perbedaan fungsi dari masing-masing action intent tersebut?

- digunakan untuk membuka atau menampilkan suatu data, misalnya membuka halaman web atau lokasi di Maps. `Intent.ACTION_DIAL` digunakan untuk membuka aplikasi telepon pada layar dialer dengan nomor yang diberikan. `Intent.ACTION_SENDTO` digunakan untuk mengirim data kepada tujuan tertentu, misalnya membuka aplikasi email dengan alamat email yang ditentukan
3. Pada kode yang digunakan, terdapat beberapa data seperti:

`data = Uri.parse("mailto:test@gmail.com")`

`data = Uri.parse("https://www.google.com")`

`data = Uri.parse("geo:-7.250445,112.768845")`

Apa fungsi data pada intent ?

- Data digunakan untuk menentukan data atau tujuan yang akan diproses oleh Intent. Contohnya, URL digunakan untuk membuka website, mailto: digunakan untuk menentukan alamat email, dan geo: digunakan untuk menentukan lokasi pada Maps.
4. Apa hubungan Action dan Data pada pilihan aplikasi yang dimunculkan oleh sistem Android ?
- Action menentukan jenis tindakan yang ingin dilakukan, sedangkan Data menentukan data yang akan diproses. Android menggunakan kombinasi keduanya untuk mencari aplikasi yang dapat menangani Intent tersebut, oleh karena itu, aplikasi yang muncul dapat berbeda tergantung pada action dan data yang diberikan.